

Contenido

1. Descripción general	2
2. Tecnologías usadas	2
3. Arquitectura y diseño	3
4. Base de datos	4
4.1. Nombre de la base de datos	4
4.2. Tabla consumptions	4
4.3. Carga del archivo CSV	5
5. Endpoints – Contrato de la API	6
5.1. GET /consumption	6
5.1.1. Parámetros de consulta (query params)	6
5.1.2. Respuesta exitosa (200 OK)	6
5.1.3. Códigos de respuesta de error	11
6. Lógica de agregación de consumos	11
7. Instalación y ejecución	13
7.1. Requisitos previos	13
7.2. Configuración de la base de datos	13
7.3. Configuración de conexión a la BD en el servicio	14
7.4. Ejecución del microservicio	14
7.5. Ejemplos de uso del endpoint /consumption	15
8. Tests unitarios	15
8.1. Archivo de tests	15
8.2. Qué validan los tests	15
8.3. Ejecución de los tests	16

Documentación técnica – Microservicio de Consumos de Energía

Autor: Oscar Javier Moreno Riveros

Fecha: 05/26/2026

1. Descripción general

El microservicio de consumos de energía tiene como objetivo exponer un endpoint HTTP que permite consultar consumos de energía de medidores de clientes, agrupados por diferentes tipos de periodos:

- Diario (daily)
- Semanal (weekly)
- Mensual (monthly)

El servicio se conecta a una base de datos relacional (PostgreSQL) que contiene los consumos históricos, cargados a partir de un archivo CSV suministrado en la prueba técnica.

Adicionalmente, el servicio incluye un campo address en la respuesta, que representa la dirección asociada al medidor. En esta versión, la dirección se obtiene mediante un valor mock fijo ("Dirección mock"), simulando la integración con un microservicio de direcciones.

2. Tecnologías usadas

- Lenguaje de programación: **Go** (Golang) – versión <1.22.x>
- Framework HTTP: **Gin** (github.com/gin-gonic/gin)
- Base de datos: **PostgreSQL**
- Driver PostgreSQL para Go: **lib/pq** (github.com/lib/pq)
- Librería estándar de Go:

- database/sql para acceso a base de datos
- time para manejo de fechas
- net/http para códigos de estado HTTP
- testing para tests unitarios

3. Arquitectura y diseño

El microservicio sigue una arquitectura sencilla de una sola aplicación:

1. El cliente (por ejemplo, Postman o curl) realiza una petición HTTP GET al endpoint /consumption, indicando:

- meters_ids: lista de IDs de medidores
- start_date y end_date: rango de fechas
- kind_period: tipo de periodo (daily, weekly, monthly)

2. El servidor:

- Valida los parámetros de entrada.
- Consulta la base de datos PostgreSQL para obtener los consumos diarios agregados por medidor y fecha.
- A partir de esos datos diarios, construye periodos (días, semanas o meses) según el parámetro kind_period.
- Suma los consumos diarios dentro de cada periodo.
- Devuelve un JSON con:
 - un array period con las etiquetas de los periodos (por ejemplo, "JUN 1", "JUN 1 - JUN 7", "JUN 2023"),
 - un array data_graph con los consumos agregados por medidor.

3. El campo address se resuelve de forma mock (no se realiza, en esta versión, una llamada real a otro microservicio).

4. Base de datos

4.1. Nombre de la base de datos

La base de datos utilizada se llama:

energy_db

4.2. Tabla consumptions

La tabla principal donde se almacenan los consumos se llama consumptions. Su definición es:

```
CREATE TABLE consumptions (
    id                      UUID PRIMARY KEY,
    meter_id               INT NOT NULL,
    active                 NUMERIC(18,6) NOT NULL DEFAULT 0,
    reactive_inductive     NUMERIC(18,6) NOT NULL DEFAULT 0,
    reactive_capacitive    NUMERIC(18,6) NOT NULL DEFAULT 0,
    exported               NUMERIC(18,6) NOT NULL DEFAULT 0,
    measured_at            TIMESTAMPTZ NOT NULL
);

CREATE INDEX idx_consumptions_meter_date
ON consumptions(meter_id, measured_at);
```

Descripción de campos:

- **id**: Identificador único de la lectura (UUID).
- **meter_id**: Identificador del medidor (entero).

- **active**: Consumo de energía activa (tipo numérico con decimales).
- **reactive_inductive**: Consumo de energía reactiva inductiva (en el CSV recibido se encuentra en 0; se deja el campo preparado).
- **reactive_capacitive**: Consumo de energía reactiva capacitiva (inicialmente 0).
- **exported**: Energía exportada (inicialmente 0).
- **measured_at**: Fecha y hora de la medición (timestamp con zona horaria).

4.3. Carga del archivo CSV

El archivo CSV suministrado en la prueba contiene 4 columnas sin cabecera, con el siguiente orden:

1. id (UUID)
2. meter_id (INT)
3. active (NUMERIC)
4. measured_at (TIMESTAMPTZ)

La importación se realizó con **pgAdmin 4**, usando la opción **Import/Export Data** sobre la tabla consumptions:

- Formato: CSV
- Opción: Import
- Header: desmarcado (el CSV no tiene cabecera)
- Delimiter: ,
- Mapeo de columnas: id, meter_id, active, measured_at

Los campos `reactive_inductive`, `reactive_capacitive` y `exported` permanecen con su valor por defecto 0.

5. Endpoints – Contrato de la API

5.1. GET /consumption

Descripción:

Obtiene los consumos de uno o varios medidores, agregados según un tipo de periodo (daily, weekly o monthly) dentro de un rango de fechas.

Método: GET

Ruta: /consumption

5.1.1. Parámetros de consulta (query params)

Nombre	Tipo	Obligatorio	Ejemplo	Descripción
meters_ids	string	Sí	1 ó 1,2,3	Lista de IDs de medidor separados por coma.
start_date	string	Sí	2023-06-01	Fecha de inicio (incluida) en formato YYYY-MM-DD.
end_date	string	Sí	2023-06-10	Fecha de fin (incluida) en formato YYYY-MM-DD.
kind_period	string	Sí	daily, weekly o monthly	Tipo de periodo para la agregación de consumos.

5.1.2. Respuesta exitosa (200 OK)

Estructura general:

```
{
```

```

"period": [
  "PERIODO_1",
  "PERIODO_2",
  "...",
],
"data_graph": [
  {
    "meter_id": 1,
    "address": "Dirección mock",
    "active": [0, 0, ...],
    "reactive_inductive": [0, 0, ...],
    "reactive_capacitive": [0, 0, ...],
    "exported": [0, 0, ...]
  }
]
}

```

- period: Array de cadenas que representan los periodos, con formatos:

- daily: "JUN 1", "JUN 2", ...
- weekly: "JUN 1 - JUN 7", "JUN 8 - JUN 14", ...
- monthly: "JUN 2023", "JUL 2023", ...

- `data_graph`: Array de objetos, uno por cada `meter_id` consultado.
 - `meter_id`: ID del medidor.
 - `address`: Dirección asociada al medidor (mock en esta versión).
 - `active`, `reactive_inductive`, `reactive_capacitive`, `exported`:
 - Arrays numéricos con la misma longitud que `period`.
 - Cada posición `i` corresponde al consumo acumulado del medidor en el periodo `period[i]`.

Ejemplo – Consumos diarios (`kind_period=daily`)

Request:

GET /consumption?meters_ids=1&start_date=2023-06-01&end_date=2023-06-10&kind_period=daily

Respuesta (estructura de ejemplo):

```
{
  "period": [
    "JUN 1",
    "JUN 2",
    "JUN 3",
    "JUN 4",
    "JUN 5",
    "JUN 6",
    "JUN 7",
```

```

    "JUN 8",
    "JUN 9",
    "JUN 10"
  ],
  "data_graph": [
    {
      "meter_id": 1,
      "address": "Dirección mock",
      "active": [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
      "reactive_inductive": [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
      "reactive_capacitive": [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
      "exported": [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
    }
  ]
}

```

Ejemplo – Consumos semanales (kind_period=weekly)

Request:

GET /consumption?meters_ids=1&start_date=2023-06-01&end_date=2023-06-26&kind_period=weekly

Respuesta (estructura de ejemplo):

```

{
  "period": [
    "JUN 1 - JUN 7",

```

```

    "JUN 8 - JUN 14",
    "JUN 15 - JUN 21",
    "JUN 22 - JUN 28"
  ],
  "data_graph": [
    {
      "meter_id": 1,
      "address": "Dirección mock",
      "active": [335.794, 376.719, 249.349, 285.905],
      "reactive_inductive": [128.48, 131.684, 104.459, 118.449],
      "reactive_capacitive": [7.614, 4.811, 11.871, 11.388],
      "exported": [6.741, 3.879, 11.094, 11.508]
    }
  ]
}

```

Ejemplo – Consumos mensuales (kind_period=monthly)

Request:

GET /consumption?meters_ids=1&start_date=2023-06-01&end_date=2023-07-10&kind_period=monthly

Respuesta (estructura de ejemplo):

```

{
  "period": [
    "JUN 2023",

```

```

    "JUL 2023"

  ],

  "data_graph": [

    {

      "meter_id": 1,

      "address": "Dirección mock",

      "active": [0, 0],

      "reactive_inductive": [0, 0],

      "reactive_capacitive": [0, 0],

      "exported": [0, 0]

    }

  ]

}

```

5.1.3. Códigos de respuesta de error

- **400 Bad Request**
 - Falta alguno de los parámetros requeridos.
 - Algún parámetro tiene formato inválido (por ejemplo, fecha mal formateada o kind_period inválido).
- **500 Internal Server Error**
 - Error al consultar la base de datos u otro error inesperado en el servidor.

6. Lógica de agregación de consumos

La lógica de agregación se compone de dos pasos principales:

1. **Agregación diaria en la base de datos (SQL)**

Para un conjunto de medidores y un rango de fechas, se ejecuta una consulta SQL que:

- Agrupa por meter_id y día (date_trunc('day', measured_at)).
- Suma los valores de active, reactive_inductive, reactive_capacitive, exported.

Ejemplo de consulta utilizada:

SELECT

```
meter_id,
date_trunc('day', measured_at)::date AS day,
SUM(active) AS active,
SUM(reactive_inductive) AS reactive_inductive,
SUM(reactive_capacitive) AS reactive_capacitive,
SUM(exported) AS exported
```

FROM consumptions

WHERE

```
measured_at >= $1
AND measured_at < $2
AND meter_id IN (...)
```

GROUP BY meter_id, day

ORDER BY meter_id, day;

2. Construcción de periodos y agrupación en Go

Con los datos diarios en memoria, una función en Go llamada `buildPeriods` genera una lista de periodos (daily, weekly o monthly), cada uno con:

- Fecha de inicio (Start)
- Fecha de fin (End)
- Etiqueta (Label) para mostrar en el JSON.

Luego, para cada día agregado y cada medidor, se determina a qué periodo pertenece y se suman los consumos en el acumulador correspondiente.

Ejemplo de firma de la función de periodos:

```
type Period struct {
    Start time.Time
    End   time.Time // exclusivo
    Label string
}

func buildPeriods(start, end time.Time, kind string) []Period
```

7. Instalación y ejecución

7.1. Requisitos previos

- Go instalado (versión <1.22.x> o similar).
- PostgreSQL instalado y en ejecución.
- Herramienta de administración de BD (por ejemplo, pgAdmin 4).

7.2. Configuración de la base de datos

1. Crear la base de datos `energy_db`.

2. Ejecutar el script de creación de la tabla consumptions mostrado en la sección 4.2.
3. Importar el archivo CSV en la tabla consumptions utilizando pgAdmin 4:
 - Opción Import.
 - Formato CSV.
 - Sin cabecera (Header desmarcado).
 - Columnas en el orden: id, meter_id, active, measured_at.

7.3. Configuración de conexión a la BD en el servicio

En el archivo main.go se define la cadena de conexión (DSN) a PostgreSQL:

```
const dsn =
"postgres://postgres:<TU_PASSWORD>@localhost:5432/energy_db?sslmode=dis
able"
```

Se debe reemplazar <TU_PASSWORD> por la contraseña real del usuario de PostgreSQL.

7.4. Ejecución del microservicio

En la carpeta del proyecto, ejecutar:

```
go run .
```

El servidor iniciará por defecto en el puerto 8080.

Se puede verificar con:

```
GET http://localhost:8080/ping
```

que devuelve:

```
{"message": "pong"}
```

7.5. Ejemplos de uso del endpoint /consumption

- Consumos diarios:

```
curl "http://localhost:8080/consumption?meters_ids=1&start_date=2023-06-01&end_date=2023-06-10&kind_period=daily"
```

- Consumos semanales:

```
curl "http://localhost:8080/consumption?meters_ids=1&start_date=2023-06-01&end_date=2023-06-26&kind_period=weekly"
```

- Consumos mensuales:

```
curl "http://localhost:8080/consumption?meters_ids=1&start_date=2023-06-01&end_date=2023-07-10&kind_period=monthly"
```

8. Tests unitarios

Se implementaron tests unitarios para validar la función `buildPeriods`, encargada de generar los periodos según el tipo (daily, weekly, monthly).

8.1. Archivo de tests

El archivo se llama:

`period_test.go`

y se encuentra en el mismo paquete que `main.go`.

8.2. Qué validan los tests

- `TestBuildPeriodsDaily`:
 - Verifica que, para un rango del 1 al 10 de junio, se generen exactamente 10 periodos diarios.
 - Comprueba que las etiquetas sean "JUN 1" y "JUN 10" para el primer y último periodo.
- `TestBuildPeriodsWeekly`:

- Verifica que, para el rango del 1 al 26 de junio, se generen 4 periodos semanales.
- Comprueba que la primera etiqueta sea "JUN 1 - JUN 7".
- TestBuildPeriodsMonthly:
 - Verifica que para un rango de junio a agosto se generen al menos 2 periodos mensuales.
 - Comprueba que el primer periodo tenga la etiqueta "JUN 2023".

8.3. Ejecución de los tests

Para ejecutar los tests unitarios:

```
go test ./...
```

El resultado esperado es algo similar a:

```
ok      <nombre-del-módulo>    0.23s
```

indicando que todos los tests pasaron correctamente.